

VERIFICA DI INFORMATICA

ARGOMENTO: PROGRAMMAZIONE A OGGETTI, UML, JAVA E JVM

I **LINGUAGGI DI PROGRAMMAZIONE AD OGGETTI**, conosciuti anche con la sigla **OOP (OBJECT ORIENTED PROGRAMMING)**, rappresentano un paradigma di programmazione che si basa sul concetto di **OGGETTO**, cioè un'entità che unisce in sé **DATI** (chiamati **ATTRIBUTI**) e **FUNZIONI** (chiamate **METODI**).

L'obiettivo dell'OOP è quello di modellare il mondo reale all'interno del software, creando programmi composti da oggetti che interagiscono tra loro proprio come le entità reali.

Questo tipo di programmazione permette di ottenere **CODICE RIUTILIZZABILE, MODULARE, FLESSIBILE** e di facile manutenzione.

I linguaggi a oggetti si fondano su quattro concetti principali: **ASTRAZIONE, INCAPSULAMENTO, EREDITARIETÀ e POLIMORFISMO**.

L'**ASTRAZIONE** consiste nel rappresentare solo gli aspetti essenziali di un oggetto, nascondendo i dettagli non importanti. In questo modo il programma risulta più semplice e chiaro.

L'**INCAPSULAMENTO** serve a proteggere i dati di un oggetto: gli attributi non sono accessibili direttamente dall'esterno, ma solo tramite metodi pubblici, detti **GETTER** e **SETTER**, che permettono di leggere o modificare i valori in modo controllato.

L'**EREDITARIETÀ** consente di creare nuove classi (dette **SOTTOCLASSI**) a partire da classi già esistenti (dette **SUPERCLASSI**). Le sottoclassi ereditano automaticamente tutti gli attributi e i metodi della superclasse, ma possono anche aggiungerne di nuovi o modificare quelli già presenti. In questo modo si evita di riscrivere codice e si favorisce il riuso.

Il **POLIMORFISMO**, infine, permette a un metodo di assumere comportamenti diversi a seconda del tipo di oggetto che lo utilizza. Ciò rende il programma più flessibile e adattabile a situazioni differenti.

Una **CLASSE** è un modello o uno schema che descrive le caratteristiche e i comportamenti comuni a un insieme di oggetti simili. In pratica, è come un "progetto" da cui si costruiscono gli oggetti.

Gli **ATTRIBUTI** rappresentano lo stato di un oggetto, cioè i suoi dati o proprietà (come il nome, l'età o il colore), mentre i **METODI** rappresentano le azioni che l'oggetto può compiere (come stampare, calcolare, muoversi, ecc.).

Un **COSTRUTTORE** è un metodo speciale che serve a inizializzare un oggetto al momento della sua creazione, assegnando valori iniziali agli attributi.

Un **OGGETTO** è invece un'ISTANZA di una classe, cioè una sua realizzazione concreta in memoria, con valori specifici per i suoi attributi.

Durante lo sviluppo di un programma orientato agli oggetti, si distinguono due fasi principali: **OOD (OBJECT ORIENTED DESIGN)** e **OOP (OBJECT ORIENTED PROGRAMMING)**.

L'**OOD** è la fase di **PROGETTAZIONE**, in cui si analizza il problema da risolvere e si stabiliscono le classi necessarie, i loro attributi, i metodi e le relazioni tra di esse. Si decide, ad esempio, quali classi devono essere collegate o quali devono ereditare da altre.

L'**OOP**, invece, è la fase di **IMPLEMENTAZIONE**, cioè quella in cui il progetto viene tradotto in codice sorgente usando un linguaggio orientato agli oggetti come **JAVA**, **PYTHON** o **C++**.

In sintesi: l'**OOD** si occupa di progettare, mentre l'**OOP** si occupa di programmare.

Per rappresentare graficamente le relazioni e la struttura di un sistema orientato agli oggetti si utilizza il linguaggio **UML (UNIFIED MODELING LANGUAGE)**.

L'**UML** è uno standard internazionale che permette di descrivere visivamente come sono organizzate le classi, gli oggetti e le loro interazioni.

Il **DIAGRAMMA DELLE CLASSI** è uno degli strumenti più importanti dell'**UML**: mostra tutte le classi del sistema, i loro attributi, i metodi e le relazioni tra esse.

Ogni classe è rappresentata da un rettangolo diviso in tre sezioni: nella parte superiore compare il **NOME DELLA CLASSE**, in quella centrale gli **ATTRIBUTI**, e in quella inferiore i **METODI**.

Tra le classi possono esserci diversi tipi di relazioni: **ASSOCIAZIONE**, quando una classe è collegata a un'altra; **COMPOSIZIONE**, quando una classe contiene una o più istanze di un'altra; ed **EREDITARIETÀ**, indicata da una freccia con triangolo vuoto che punta verso la superclasse.

Il linguaggio **JAVA** è uno dei linguaggi orientati agli oggetti più diffusi al mondo. È stato creato negli anni Novanta da **SUN MICROSYSTEMS** (oggi di proprietà **ORACLE**) e si basa sul principio "**WRITE ONCE, RUN ANYWHERE**", cioè "scrivi una volta, esegui ovunque".

Questo significa che un programma scritto in Java può essere eseguito su qualsiasi sistema operativo, purché sia installata la **JVM (JAVA VIRTUAL MACHINE)**, senza bisogno di modifiche al codice.

JAVA è un linguaggio **PORTABILE**, **SICURO** e **ROBUSTO**. Offre una gestione automatica della memoria tramite il **GARBAGE COLLECTOR**, che libera lo spazio non più utilizzato dagli oggetti.

Inoltre, mette a disposizione una **LIBRERIA STANDARD** molto ampia, con migliaia di classi per la gestione di file, rete, grafica, database, interfacce grafiche e molto altro. Tra i principali **VANTAGGI** di Java troviamo la portabilità, la sicurezza, la stabilità e la facilità di manutenzione.

Gli **SVANTAGGI**, invece, riguardano le prestazioni leggermente inferiori rispetto ai linguaggi compilati nativamente (come **C** o **C++**) e la sintassi piuttosto verbosa, che richiede di scrivere più codice rispetto ad altri linguaggi moderni.

La **JVM (JAVA VIRTUAL MACHINE)** è il componente che rende Java portabile e indipendente dalla piattaforma.

Quando un programma Java viene compilato, il file sorgente con estensione **.java** non viene trasformato in un file eseguibile per un sistema operativo specifico, ma in un file **.class** contenente **BYTECODE**, un linguaggio intermedio che può essere interpretato su qualsiasi macchina.

La **JVM** legge questo bytecode e lo traduce in tempo reale nelle istruzioni del sistema su cui il programma viene eseguito.

All'interno della **JVM** troviamo diverse **AREE DI MEMORIA** con ruoli specifici.

I **REGISTRI** contengono informazioni sull'esecuzione, come il **PROGRAM COUNTER**, che indica quale istruzione deve essere eseguita.

Lo **STACK** memorizza i **FRAME DEI METODI** in esecuzione, che contengono le variabili locali e i dati temporanei.

L'**HEAP** è l'area in cui vengono creati gli **OGGETTI** e gli **ARRAY**, e viene gestita automaticamente dal **GARBAGE COLLECTOR**.

L'**AREA DEI METODI** contiene il **BYTECODE DELLE CLASSI** caricate, insieme ai metodi statici e alle costanti.

Infine, la **NATIVE METHOD AREA** è una sezione dedicata all'esecuzione di metodi scritti in altri linguaggi, come C o C++.

La **CREAZIONE DEGLI OGGETTI** in Java avviene solitamente all'interno del **METODO MAIN**, che rappresenta il punto di ingresso del programma.

Quando si utilizza la parola chiave **NEW**, la JVM alloca memoria nell'HEAP, chiama il **COSTRUTTORE** della classe per inizializzare l'oggetto e restituisce un **RIFERIMENTO** che viene assegnato a una variabile.

Per esempio, se abbiamo una classe "Auto", possiamo scrivere:

```
Auto a1 = new Auto("Fiat", 2020);
```

In questo modo viene creato un oggetto "a1" di tipo "Auto", con i valori specificati, e sarà poi possibile richiamare i suoi metodi, come a1.accendi();.

In conclusione, la **PROGRAMMAZIONE ORIENTATA AGLI OGGETTI** è un modello di sviluppo potente e moderno, che permette di scrivere programmi più ordinati, chiari e vicini al modo in cui pensiamo la realtà.

Concetti come **CLASSE**, **OGGETTO**, **EREDITARIETÀ**, **INCAPSULAMENTO** e **POLIMORFISMO** sono alla base della progettazione software attuale.

L'uso dell'**UML** consente di rappresentare graficamente la struttura dei programmi prima della loro realizzazione, mentre **JAVA**, grazie alla **JVM**, permette di scrivere applicazioni portabili, sicure e versatili, utilizzate in tutto il mondo, dai sistemi aziendali alle app mobili.